

Loopy Build Log

A record of the work behind Loopy's move from a local prototype to a deployed product — the database migration, the auth rebuild, the performance collapse and its cause, the security audit, and every bug worth remembering.

Where the stack landed

Two of these layers were replaced outright during this stretch.

LAYER	WAS	NOW	WHY IT CHANGED
Database	SQLite (local file)	Neon Postgres	Had to be reachable from a deployed backend
Auth	Supabase	Google Identity Services + email/password	Product decision — drop the dependency
Backend	NestJS, local only	NestJS on Vercel serverless	First deployment
Frontend	Next.js 15 on Vercel	unchanged	—
Email	none	Gmail SMTP via nodemailer	Signup codes and order notifications

1 · Database migration

Moving from a local SQLite file to hosted Postgres. The schema port was trivial; everything that went wrong was about the network between the app and the database.

Real store data was left behind in SQLite

DATA LOSS RISK

SYMPTOM	After migrating, the storefront rendered — but the seller's actual store, <code>cpaybara</code> , was missing. Only the four seeded demo stores existed.
CAUSE	The migration pushed the <i>schema</i> to Neon and ran the seed script. Nobody moved the <i>data</i> . The real store — custom layout, uploaded logo, <code>published: true</code> — was still sitting in <code>backend/prisma/dev.db</code> .
FIX	Dumped every table to JSON, coerced the type differences (SQLite stores dates as epoch integers and booleans as 0/1), and reinserted into Neon preserving primary keys so every foreign key stayed intact.
EVIDENCE	6 users, 5 sellers, 16 products, 4 visits, 1 dispute migrated. <code>/sellers/cpaybara</code> returned the store published, with logo and layout config intact.

Why it nearly slipped through: the app looked correct after migrating, because seed data filled the same shapes as real data. The failure was only visible to someone who knew which store *should* have been there.

Prisma's 5-second transaction timeout broke checkout

HIGH

SYMPTOM	Placing an order returned <code>500 Internal Server Error</code> , consistently.
CAUSE	<code>Transaction already closed: timeout was 5000 ms, however 6816 ms passed</code> . Checkout runs several statements in one interactive transaction. On SQLite each was microseconds; against a remote — and possibly auto-suspended — Neon compute, each is a network round trip.
FIX	Raised every <code>\$transaction</code> to <code>{ timeout: 20000, maxWait: 15000 }</code> , and moved the seller notification <i>outside</i> the transaction so a slow insert can't roll back an order that was already paid for.

2 • Authentication rebuild

Supabase removed entirely, replaced with Google ID-token verification plus a self-hosted email/password flow.

What replaced what

FLOW	ENDPOINT	MECHANISM
Seller / admin sign-in	POST /auth/google	Google ID token verified server-side, aud pinned to the client ID
Shopper sign-in	POST /stores/:u/customer-auth/google	Same verification, scoped per store
Shopper signup	.../register → .../verify	6-digit emailed code; account created only after it checks out
Shopper login	.../login	bcrypt; identical error for wrong password and unknown email

Supabase sent a magic link when a code was wanted

DIAGNOSIS ONLY

- SYMPTOM

"Email me a code" delivered a clickable login link instead of a 6-digit code.
- CAUSE

Not a code bug. signInWithOtp was already called correctly with no redirect. Supabase decides link-vs-code purely from the email template: {{ .ConfirmationURL }} sends a link, {{ .Token }} sends a code — and two templates are involved, Magic Link for existing users and Confirm Signup for new ones.
- OUTCOME

Moot — Supabase was removed shortly after. The signup code is now generated and emailed directly, bcrypt-hashed at rest, 10-minute expiry, capped at 5 attempts.

Worth stating plainly: the first diagnosis here was wrong — one template was blamed when two were responsible. Correcting it cost a round trip.

An unverified email could occupy an account

DESIGN FIX

- RISK

If signup wrote the Customer row and then asked for verification, anyone could register someone else's email and hold that account hostage.
- FIX

Signup writes to a separate PendingSignup table; the real account is created only once the code is verified. Pending rows live in the database rather than in memory, so codes survive a restart and work across serverless instances.

3 · The performance collapse

Storefront loads went from instant to 16–25 seconds after the Neon migration. Two independent causes, neither visible without measuring.

STOREFRONT, BEFORE	STOREFRONT, AFTER	API PAYLOAD	PER QUERY
16-23 s	~1 s	3.4 → 0.9 MB	1442 → 296 ms

Neon's connection pooler cost 1.1 s on every single query

CRITICAL

SYMPTOM	Every page slow, in proportion to how many queries it ran.
DIAGNOSIS	Timed a bare <code>SELECT 1</code> against both endpoints on a warm connection — isolating the network from payload size and from Prisma.
MEASURED	Pooled (<code>-pooler</code>): 1442 ms · Direct: 296 ms. PgBouncer was adding roughly 1.15 s per statement.
FIX	Point both <code>DATABASE_URL</code> and <code>DIRECT_URL</code> at the direct endpoint. The pooler exists for serverless runtimes that open a connection per invocation; a long-lived Nest process with Prisma's own pool gains nothing and pays the tax.

This reverses on deploy. The backend now runs on Vercel serverless, where every request is a separate invocation. Production uses the pooled endpoint with `pgbouncer=true&connection_limit=1`; pointing it at the direct host there would exhaust Neon's connection limit under load.

Images are stored as base64 inside Postgres rows

ARCHITECTURAL

SYMPTOM	One storefront's API response was 3.4 MB and took 25 s.
CAUSE	Uploads are kept inline as data-URLs in <code>Seller.logoUrl</code> , <code>Seller.storeConfig</code> and <code>Product.images</code> . A single store held 3.3 MB — a 710 KB logo, 985 KB of embedded layout images, 1.59 MB of product photos. The uploader's own comment read "original quality, no resize". On local SQLite this was a disk read; over the network it is the entire payload.
FIX	Downscale and re-encode to WebP in the browser before storing (max 1600 px, quality 0.82), and re-encode the existing rows once with <code>sharp</code> .
EVIDENCE	3007 KB → 600 KB, an 80% reduction , with no visible quality change at display sizes.

Still the weakest part of the design. Compression bought headroom; it didn't fix the model. Images belong in object storage with URLs in the database. This will resurface as the catalogue grows.

Front-end causes, found alongside

MEDIUM

TWO DEV SERVERS	Two <code>npm run dev</code> instances on one folder, both compiling into the same <code>.next</code> . Symptom: <code>ENOENT _buildManifest.js.tmp</code> and route timings swinging 1–4 s.
NO LOADING STATES	Not a single <code>loading.tsx</code> existed, so a click left the <i>previous</i> page on screen until the next was ready — reading as "the button did nothing". Added route skeletons.
REFETCH EVERY VISIT	Every page is a client component fetching on mount, so Dashboard → Orders → Dashboard was three round trips. Added a 15 s GET cache with in-flight de-duplication.
LAYOUT THRASH	The landing page's magnetic buttons called <code>getBoundingClientRect()</code> on every mousemove — a forced reflow per event. Now measured once on enter and coalesced to one update per frame.

4 · Order lifecycle

The fulfilment flow barely existed. Two bugs meant core paths had never worked at all.

Every authenticated storefront call sent `Bearer undefined`

CRITICAL

SYMPTOM	Checkout failed with "Unauthorized" while the header clearly showed the shopper signed in.
CAUSE	The API returns the JWT as <code>accessToken</code> ; the client read <code>c.token</code> . The value was <code>undefined</code> , so every request sent <code>Bearer undefined</code> . Being "signed in" was purely a localStorage display — the token never reached the server.
AGE	Pre-existing. The old Supabase code had the identical mismatch, which means checkout had never once worked.
FIX	Normalise at the storage boundary, and accept either shape when loading — so sessions already saved in browsers heal themselves instead of forcing a sign-out.

Checkout links pointed at a route that doesn't exist

CRITICAL

SYMPTOM	None visible to the seller — the link generated and copied fine.
CAUSE	Links were built as <code>/product/<id></code> . There is no such route; storefront products live under <code>/s/<store>/product/<id></code> . Verified: 404 . Every checkout link ever shared was dead.
FIX	Rebuilt as a multi-product basket generating <code>/s/<store>/cart?add=<id>:<qty>,...</code> . The cart resolves the IDs and fills itself.
DESIGN NOTE	The link carries IDs and quantities only, never prices — they resolve from the live catalogue on open, so a tampered link can't change what something costs. Quantities are <i>set</i> rather than <i>added</i> , so tapping a WhatsApp link twice doesn't double the order.

What the lifecycle supports now

CAPABILITY	DETAIL
Accept / reject	Rejection restores stock, records the reason, emails the buyer
Ship	Courier name required; tracking number auto-generated; both emailed and shown to the buyer
Undo	Steps back one stage, clearing tracking number and courier when leaving Shipped
Buyer cancellation	Reason required — preset list or free text; notifies the seller instantly
Cancelled vs Rejected	Separate filters; <code>cancelledBy</code> distinguishes them, historical rows backfilled
COD payment state	Reads "due" until delivered, then "Paid" — derived, never stored, so it can't drift

Why COD needed derived state: a COD order is created with `status: 'Paid'` so it enters the seller's queue. Showing that verbatim made an order the seller hadn't even accepted look already paid. Payment state is now computed from `paymentId` + `status` rather than stored separately.

5 • Security audit

A full audit, run on request. Every finding below was confirmed by exploiting it against a running server, then re-tested after the fix.

A hardcoded master OTP granted admin to anyone

CRITICAL

FOUND VIA A routine question — "what are the admin credentials?" The admin account had **no password at all**, meaning the admin login page could never have worked. The only way in was the backdoor.

THE HOLE `auth.service.ts` accepted `0000` as a master OTP for *any* phone number. The seeded admin's phone is `9000000000`. Posting both to the **live production API** returned a full admin token — no password, no secret.

FIX Gated behind `NODE_ENV !== 'production'`, which every host sets. Local dev unchanged; production rejects it. Also stopped `requestOtp` echoing the code in its own response, which made the OTP meaningless.

Customer PII readable with no token

CRITICAL

THE HOLE `GET /orders/:id` had no guard. Retrieved a real order unauthenticated and got the buyer's name, phone and full delivery address.

REASONING Order IDs are cuids, but unguessability is not access control.

FIX Guard plus an ownership check: the buyer who placed it, the seller fulfilling it, or an admin. Nobody else.

Full findings

FINDING	SEVERITY	BEFORE	AFTER
Master OTP → admin token	CRITICAL	admin granted	rejected in prod
Read any order, no token	CRITICAL	200 + full PII	401
Four unguarded write endpoints	CRITICAL	200	401
Cross-tenant order access	HIGH	200	403
No rate limiting anywhere	HIGH	unlimited	429 after 8/min
Stored XSS via store config	MEDIUM	executes	neutralised

The four unguarded writes let anyone mark an order delivered (releasing escrow), post fake reviews, or open disputes on *any* order. None were used by the interface — they were legacy from an older marketplace flow, so guarding them broke nothing.

What the audit found clean: no `dangerouslySetInnerHTML` anywhere; admin endpoints already enforced role checks in the service layer; Prisma parameterises all queries; bcrypt at 10 rounds; the signup code was already hashed and attempt-capped.

One first impression was wrong and corrected by testing: the admin controller *looked* unguarded, but a seller token returned `403` — the check lives in the service, not the controller.

6 • Deployment

Frontend and backend as separate Vercel projects, on a custom domain.

Domain move between Vercel accounts stalled on verification

MEDIUM

SYMPTOM	Both domains stuck on "Verification Required"; the live site returned <code>DEPLOYMENT_NOT_FOUND</code> throughout.
CAUSE	Two layers. Removing the domain from the old <i>project</i> isn't enough — Vercel also holds it at <i>account</i> level. And each domain entry needs its own TXT value: verifying the apex does not verify <code>www</code> .
CORRECTION	The first advice given here — that one TXT record would cover both — was wrong, and cost several attempts before it was caught.
RESULT	<code>www</code> serves 200, apex 308-redirects to it, DNS untouched throughout.

Deployed latency, measured

VERIFIED

RESULT	Warm request $\approx$ 0.6 s end to end from India. <code>X-Vercel-Id: bom1::iad1</code> — the request enters at Mumbai's edge, the function runs in Virginia beside the database.
IMPLICATION	The 296 ms per-query cost measured locally was laptop-to-Ohio. Co-located in production it's negligible; the remaining latency is the India $\leftrightarrow$ US hop and payload size, not the database.
CONSEQUENCE	Paying to disable Neon's auto-suspend would buy almost nothing — which was the plan before measuring.

7 • Interface bugs worth recording

Two were CSS specificity traps that will recur if the cause isn't remembered.

Typed text was invisible in the variant editor

MEDIUM

SYMPTOM	Typing into a variant's label field showed nothing at all.
CAUSE	<code>.c-input</code> is declared inside <code>@layer utilities</code> with <code>w-full</code> . Tailwind emits custom utilities <i>after</i> its core ones, so <code>width:100%</code> silently beat the <code>w-24</code> / <code>w-20</code> on the neighbouring fields. Those stretched full-width, and the <code>flex-1</code> label — with <code>flex-basis: 0</code> — collapsed to a few pixels.
FIX	Move the widths onto a CSS grid, where a class on the child can't override the column.

Recurring trap: any width utility applied to `.c-input` is ignored. Use a grid, a wrapper, or Tailwind's `!` modifier.

Form fields misaligned inside a grid row

LOW

CAUSE The shared `Field` component carried `mt-4 first:mt-0`. Inside a three-column grid, `first:` applied only to the first cell — the other two each got a top margin and sat lower.

FIX Remove self-spacing from the component; let the parent's `gap` own it.

Preview button felt dead, and could be blocked entirely

MEDIUM

CAUSE It awaited two network round trips — one re-uploading a 710 KB base64 logo — *before* calling `window.open`. Past an `await`, that call sits outside the click's stack, so browsers can silently block the popup.

FIX Open the tab synchronously, save in the background. Nothing is lost: preview reads the draft from `localStorage`, which is written on every edit.

The supplied logo was invisible on the app's background

LOW

CAUSE Sampling the PNG showed the wordmark at `#F0F0F0` — near-white, on a `#F6F5F0` page.

FIX Generated a navy-wordmark variant for light surfaces, keeping the original for dark ones. The green mark is untouched in both.

Still open

Known and deliberate. Nothing here is a surprise.

- **Email isn't sending.** All four templates are built and firing; they need a Gmail app password in `SMTP_USER` / `SMTP_PASS`. Until then signup codes fall back to being returned in the API response — fine locally, **not acceptable in production**.
- **Payments are stubbed.** No real gateway; orders are marked paid without money moving.
- **Images live in the database.** Compression bought headroom, not a fix.
- **Credentials need rotating.** Several were exposed during setup and should be cycled before real traffic.
- **Weak admin password** on a panel that can see every seller, order and payout.
- **CORS is open** (*). Low risk — auth is a bearer token, not cookies — but worth narrowing.
- **Preview deploys can't use Google sign-in.** Vercel generates a new hostname per deploy and Google allows no wildcards. Expected, not a fault.

Worth remembering

- **Measure before optimising.** The pooler finding and the "is auto-suspend the problem?" question both reversed on measurement. Acting on the pre-measurement plan would have paid for the wrong thing.

- **The same setting flips between local and serverless.** A direct connection wins for a long-lived process; the pooler is mandatory for serverless. Same database, opposite answers.
 - **Unguessable is not access control.** Cuid order IDs were doing the job of an authorisation check.
 - **Two features had never worked.** Checkout and checkout links both failed silently — no error, no complaint, just nothing. Worth testing the paths nobody has reported on.
 - **Derive state that can drift.** COD payment status and cancellation attribution are computed from what's already stored, so they can't fall out of sync.
 - **Migration means schema *and* data.** Seed data made an incomplete migration look complete.
-

Loopy engineering log · August 2026 · Internal. Contains no credentials, connection strings or keys — those live only in environment files and the separate credentials reference.